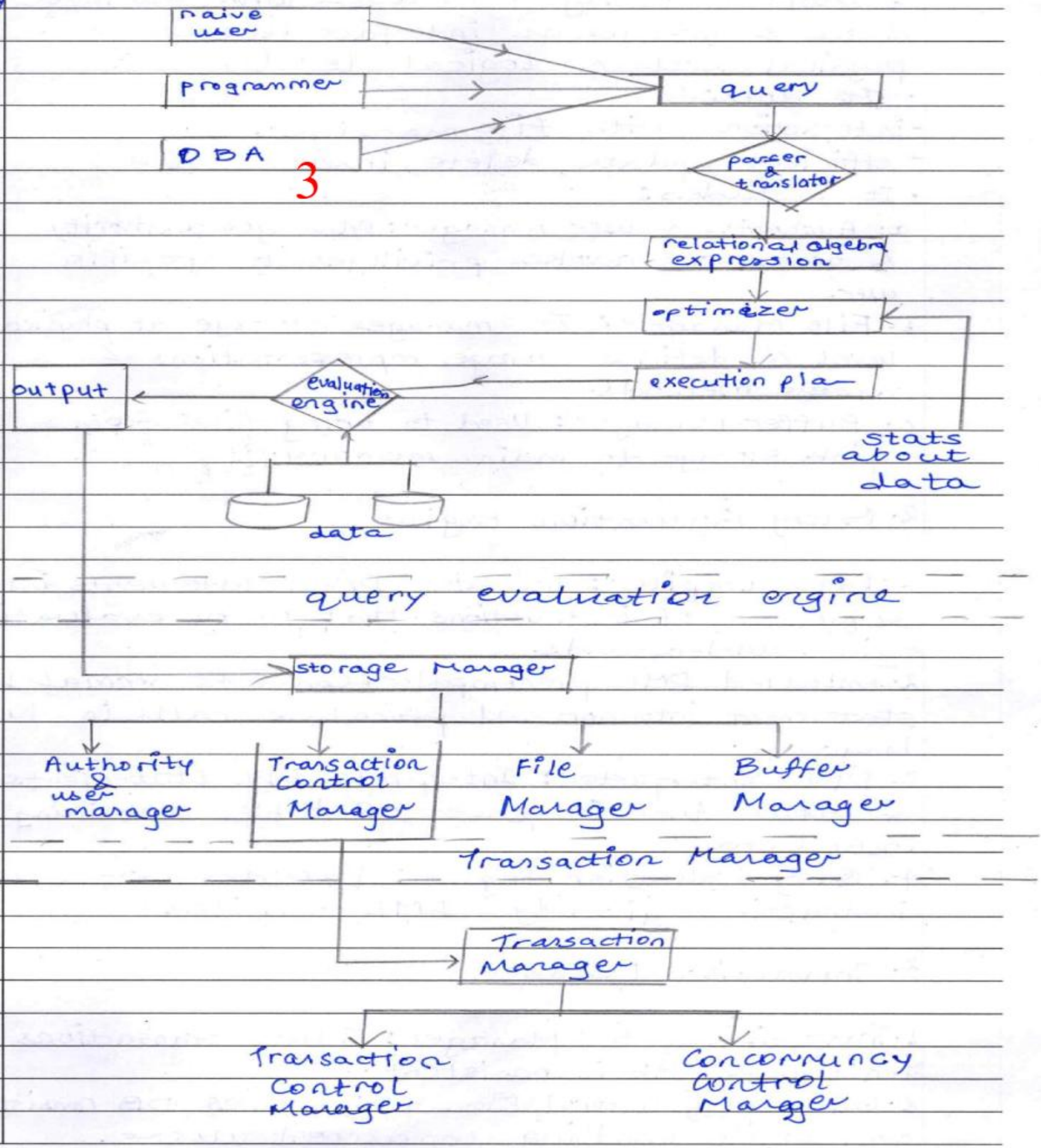
Q.1 A)


naive user
programmer
DBA
query
parser & translator
relational algebra expression
optimizer
execution plan
stats about data
output
evaluation engine
data
query evaluation engine
storage Manager
Authority & user manager
Transaction control Manager
File Manager
Buffer Manager
Transaction Manager
Transaction Manager
Transaction control Manager
Concurrency control Manager

1. Storage Manager: It deals with low level data & acts as an interface between physical level & logical level
• It does:
- interaction with file manager.
- efficient update, delete, insert
• It includes:
a. Authority & user manager: Manages authority & grants or revokes privileges to specific user.
b. File Manager: It manages storage at physical level & defines storage representations & access methods.
c. Buffer Manager: Used to bring file from disk storage to main memory.

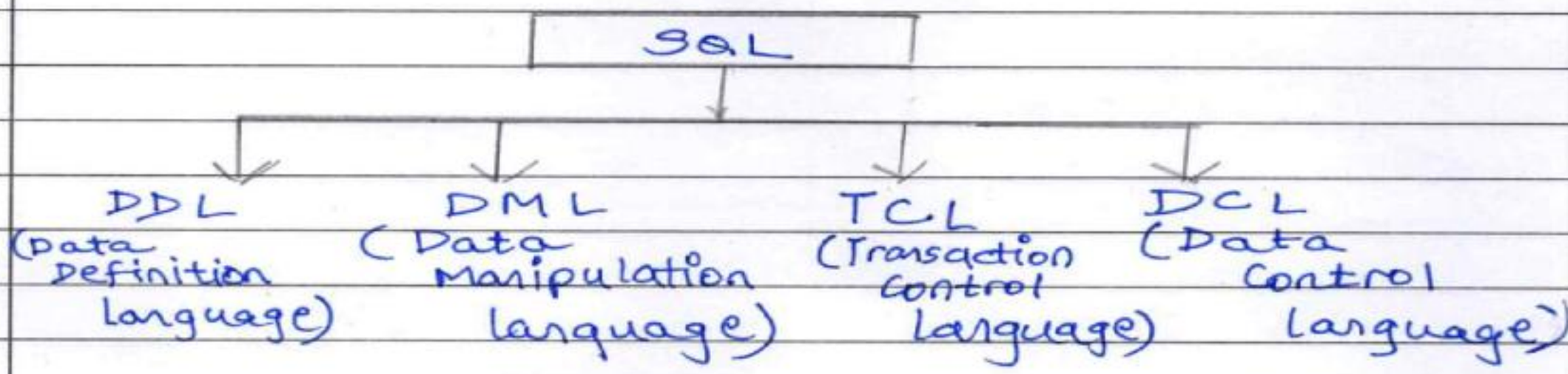
2. Query Evaluation Engine

1. DML Compiler: Translates DML statements into sequences of instructions that query evaluation engine understands.
2. Embedded DML precompiler: Converts normal DML statement into normal procedure calls in host language.
3. DDL Interpreter: Interprets DDL statements & records them in a set of tables containing metadata.
4. Query Evaluation Engine: Executes low-level instructions given by DML Compiler.

3. Transaction Manager

1. Transaction Control Manager: Ensures transactions don't make dB inconsistent.
2. Concurrency control Manager: Ensures DB consistency even while handling concurrent users.

Q.2 A



- SQL is divided into 4 components which perform each of the responsibility of an RDBS.

3

1. DDL

- Data Definition language is used to create database schema, structure & defines the rules, primary keys & data types.
- It makes a framework to store the data.
- DDL operations include

~~1. TRUNCATE;~~

1. DROP: drops & deletes dB schema.

Syn : DROP TABLE table_name ;

2. ALTER: Used for modification of schema

Syn : ALTER TABLE table_name ADD Email char(30);

3. CREATE: Used for creating table & database

Syn: CREATE TABLE student(name varchar(30));

2. DML

- It is used to manipulate data & values inside the database.
- Used to insert, update, delete values.

1. SELECT : Used for retrieval

Syn : SELECT * FROM student ;

2. INSERT : Used for adding values

Syn : INSERT INTO student(name) values("Mohan");

3. TCL

- It is used to preserve consistency in the database & keep the data durable.
- SOME COMMANDS used:

a. SAVEPOINT savepoint_name; : Used to create a snapshot of the table to be rolled back to.

b. ROLLBACK to savepoint_name; : Used to rollback to a consistent state of DB.

c. COMMIT; : Used to COMMIT changes.

By default the changes are not committed until the current ongoing session is ended.

4. DCL

- Used to control authorization to users

Ex. CREATE user user_name;
GRANT ALL priveliges ON user_name;
REVOKE ALL privileges ON user_name;

- GRANT : Used to apply privileges & permissions.
- REVOKE : Used to take back permissions.

B.

```
CREATE TABLE Employee (
 EmpID ,1
  Name,
 DeptID,
 Salary);

CREATE TABLE Department (
 DeptID,
 DeptName,
 Location );

SELECT  Name, Department. DeptName,
 salary FROM Employee
  INNER JOIN
   DEPARTMENT
    ON
     Employee. DartmentID= Department. DeptID-
```

3A

- Edgar Codd proposed 12 rules for an ideal RDBMS.

1. Information Rule: All the information in a relational database must be represented explicitly at logical level & in exactly one way, by values & rows & cells in tables.

Ex:

St_ID	St_Phone
1	9xxxxxx60
	9xxxxxx79

} Violates!

St_ID	St_Phone
1	9xxxxxx60
1	9xxxxxx79

} Correct !

2. Guaranteed access Rule: Each & every datum (atomic value) must be logically accessible resorting to a combination of table name, primary-key & column name.

Ex: SELECT student_name FROM student
WHERE Student_ID = 1;

3. Systematic treatment of null values: Null values are supported in a fully RDBMS for representing missing information or inapplicable data in a systematic way, independent of data type.

4. Dynamic on-line catalog based on relational language: The database description should be represented at the logical level in the same way as ordinary data, so that the authorized users can use the same relational language to it's interrogation the same way as they do to the regular data.

5.

6. View updating Rule: All the views which are theoretically updatable are also updatable by the system.

7. High level insert, update, delete: The capability of handling base relation & derived relation as a single operand is not only applicable to the retrieval of data but also applies to updation, deletion & insertion.

8. ~~Logical~~ Physical level Independence: Application programs & terminal activities remain logically unimpaired whenever ~~that~~ data is changed in storage representations or access methods.

9. Logical level Independence: Application programs & terminal activities remain logically unimpaired when information-preserving changes are of a kind that theoretically permit un-impairment are made to the base table.

10. Integrity Independence: Integrity constraints specific to a database should be definable in a relational data sub-language & storable in catalog, not in application programs.

11. Distribution Independence: The data manipulation sub-language should enable application programs & queries to remain logically unchanged whether & whenever the data is physically

distributed or compressed/compiled/clustered 1.5

12. Data subversion rule : If a system provides low-level interface, the it should not be able to subvert the security constraints.
Ex: User should not be able to bypass the security.

B

- Database anomalies are problems & bugs that may lead a database to be inconsistent.
- Removing these anomalies makes the database ~~for~~ durable & removes redundancy.
- let's take an example:

ID	Name	Dept	Dept-ID
1	Yash	ECE	1C
2	Yash	CSE	2C
3	Mohan	ME	3C
4	Soham	IT	4C
5	Narendra	IT	4C
6	Amit	CSE	2C
7	Shubhendu	ECE	1C

— 3BI

i) Insertion anomaly

It arises from the fact that for a DB to be good & normalized, no value should be null but sometimes some column needs a value but the corresponding column information is missing.

Ex: In table (3BI) the departments are namely ECE, CSE, ME, IT.

Suppose the college inaugurates a new dept. AI∩DS, but the enrollment for it hasn't started. In that case the table will look like: 3

ID	Name	Dept	Dept_ID
Null	Null	AI∩DS	5C

This violates good database design & isn't in 1NF form. Therefore to solve it, decome (3BI)

ID	Name	Dept_ID
some value	Not Null	Not Null

(3DII)

Dept_ID	Dept
1C	ECE
2C	CSE
3C	ME
4C	IT
5C	AI∩DS

(3D III)

Now AI∩DS Dept can exist without violating any rule.

ii) Update

It arises due to redundancy & system crashes/anomalies.

Ex: Suppose College ~~branch~~ changes Dept name "CSE" to "CE".

∴ If while updating, if some values change but some do not, ambiguity & inconsistency arises.

ID	Name	Dept	Dept_ID
2	Yash	CE	2
6	Amit	CSE	2

} Question arises which one is correct.

Therefore to ~~reduce~~ remove the anomoly, redundancy should be reduced i.e. decompose tables.
It will be same as (3D II) & (3D III)

iii) Deletion anomoly
It arises when there is a single value of a column & it gets deleted leaving ambiguity -
Ex.
In table (3D I) If ➤ ID = 3 is deleted then the whole "ME" dept. will be deleted but that is not the case.
Therefore table should be decomposed like (3D I) & (3D III)

ID	Name	Dept	Dept ID
2	Yash	CSE	2C
3	Mohan	ME	3C
4	Soham	IT	4C

X

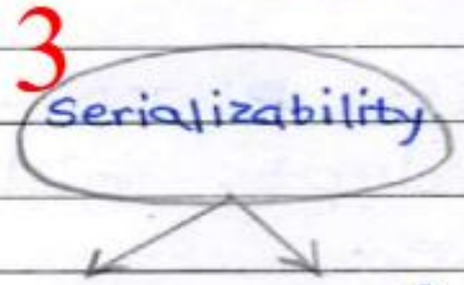
Dept_ID	Dept
1C	ECE
2C	CSE
3C	ME
4C	IT
5C	AI.DS

Even if ID = 3 is deleted, the entry of ME Department still remains.

Q.4A

- Serializability is a property of checking whether a non-serial schedule is serializable or not i.e. can a parallel schedule with interleaved transactions can be converted into a serial schedule or not.

- Categories of Serializability in DBMS.



3

* Conflict Serializability

A schedule is conflict serializable if it has any of these conflict pairs.

Conflict Pairs	Non conflict pair
$R_1(A) : W_2(A)$	$R_1(A) : R_2(A)$
$W_1(A) : R_2(A)$	$W_1(A) : R_2(B)$
$W_1(A) : W_2(A)$	$W_1(A) : W_2(B)$

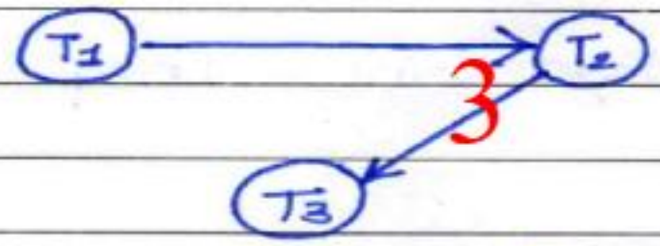
let's take a schedule :

T_1	T_2	T_3
$R_1(A)$		
	$W_2(A)$	
	$R_2(A)$	
		$W_3(A)$

Conflict Pairs Found:

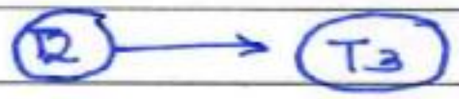
$R_1(A) : W_2(A)$, $R_2(A) : W_3(A)$

∴ To find out the order of precedence, we draw a precedence graph with nodes representing transactions



For node T_1; indegree (edges coming towards it) is 0, ∴ Take it out of the graph

Now,



T_2 indegree = 0, take it out

T_3 ← indegree = 0, take it out

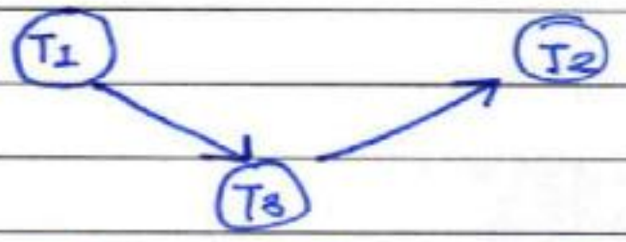
∴ Precedence order is

$T_1 \rightarrow T_2 \rightarrow T_3$

∴ when the schedule will be converted into ~~$T_3 \rightarrow T_2 \rightarrow T_1$~~. $T_1 \rightarrow T_2 \rightarrow T_3$

another ex.

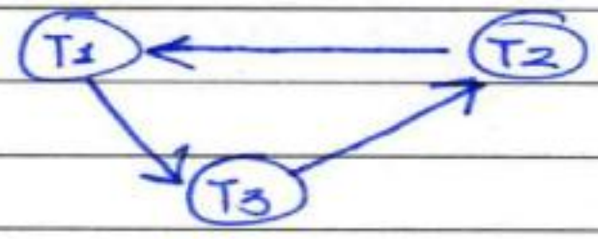
T_1	T_2	T_3
$R_1(A)$		
		$W_3(A)$
	$R_2(A)$	
	$W_2(A)$	



This will execute serially as $T_1 \rightarrow T_3 \rightarrow T_2$

another ex.

T_1	T_2	T_3
$R_1(A)$		
		$W_3(A)$
	$R_2(A)$	
	$W_2(A)$	
$W_1(A)$		



In the precedence graph, loop is formed ∴ not conflict serializable.

If loop is formed can't say if schedule is serializable or not. Will have to check for view serializability.

Conflict serializab[redacted] when loop is not formed.

C

* database consistency is achieved by ACID properties.

1. Atomicity : It states that ~~only one~~ the transactions should either fully complete or not at all complete. Only if there is surety then only changes to DB should be made.
Ex.
Read A = $500
A = 500 + 100 $
Write A

The above transaction should complete with every instruction or abort. If changes are made haphazardly, DB may remain in an inconsistent state.

2. Consistency : It states that the data should always be consistent & there should be no errors in it.

Ex.

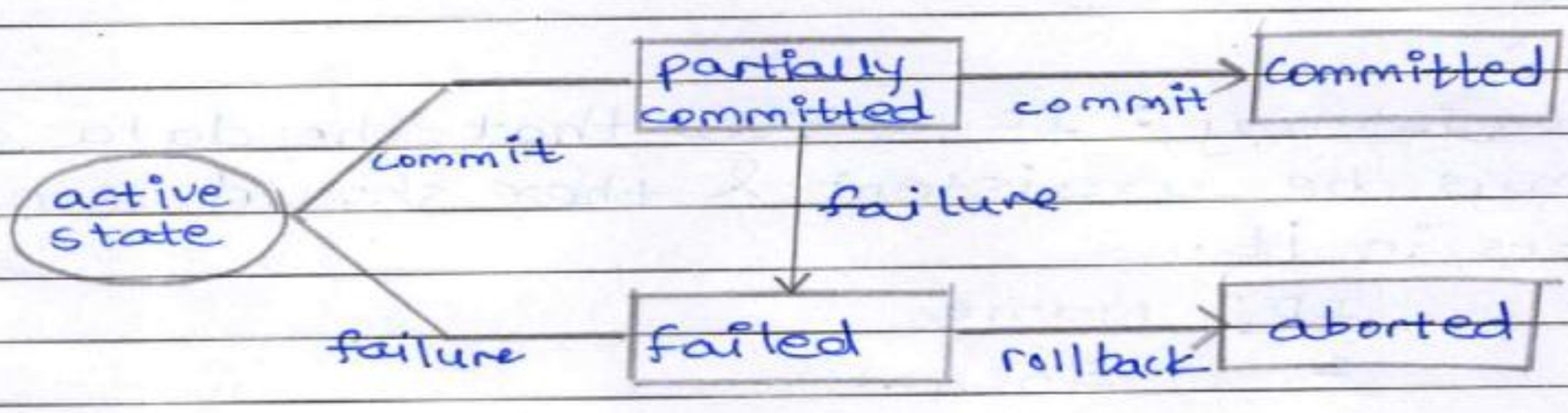
ID	Name	
1.	Snehal	
2	Soham	} inconsistent data
2	Singham	} Not allowed

3. Isolation : States that each data & transaction should be seperated & isolated from each other & should not make any changes to any other data.

4. Durability : States that despite system failure, hardware failures, the database should remain consistent & all the commits made to DB should stay as they are.

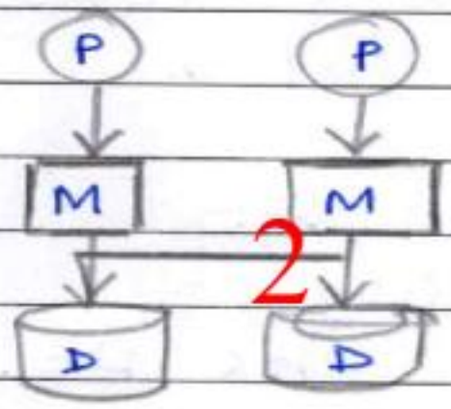
- Different states of transactions are:

- active state: The transaction is ongoing & the computations are taking place.
- partially committed State: Intermediate step between active & committed state checks for inconsistencies & any anomalies
- committed state: Changes are made to the database.
- failed state: The transaction fails due to inconsistency, conflicts & deadlocks.
- aborted: The transaction is halted & terminated & the process begins again.



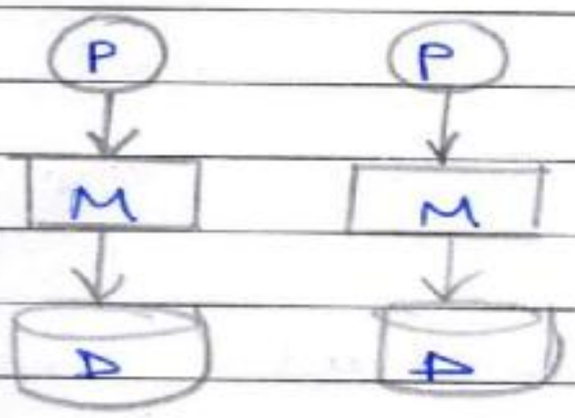
Q.5A

★ Shared Memory



It states that the memory & processing of the system is shared.

★ Shared Nothing



It states that nothing is shared & each program works on it's different processing & different database.

C.

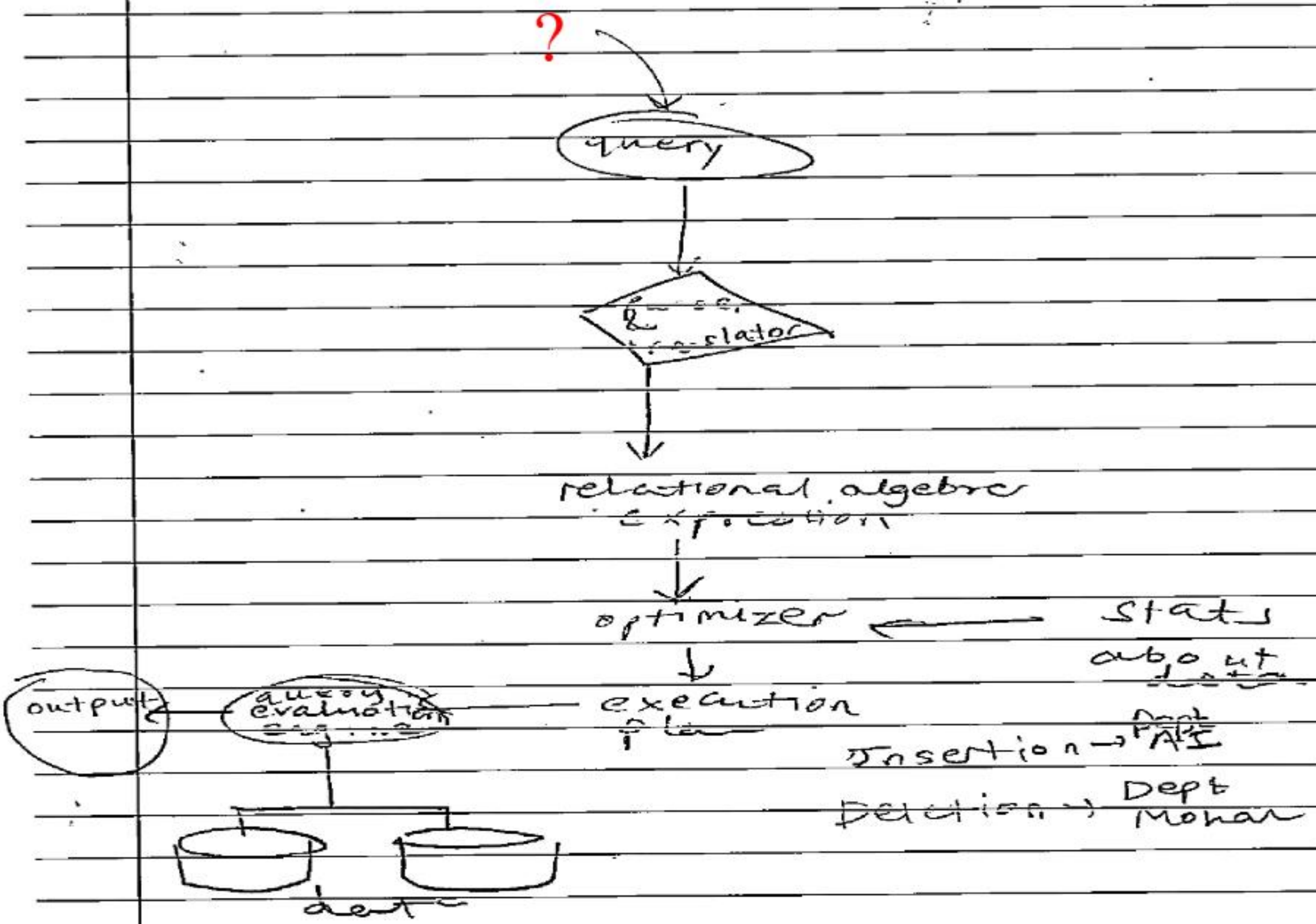
CAP

1. Consistency : Consistency of a database is preserved and all the updates are reflected in the DB.
2. Avalabilty : Availability of DB is achieved through compromising the accuracy & consistency of the DB.
3. Partition Tolerance : It states that which system needs how much consistency & how much availability.

Ex.

BASE

- Basically available : States that availability is very high.
- soft state : Data may change & inaccuracies are common.
- Eventually consistent : States that after a given time the DB will be fully consistent.



St id	St name	Dept
1	Yash	ECE
2	Yash	CSE
3	Mohan	ME
4	Soham	IT
5	Narendra	IT
6	Amit	CSE
7	Amisha	ECE